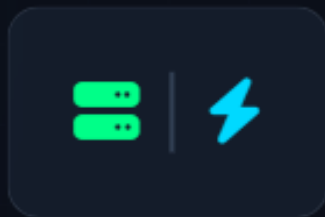


200 OK



```
@app.get("/")
```

Backend Development with **FastAPI**

From Fundamentals to AI Integration 🚀

```
async def root():
```

>_ TA Ahmed Mohamed & TA. Ahmed Mostafa | Section 2

```
{"msg": "Hello"}
```

What is the Backend?



The Engine Under the Hood

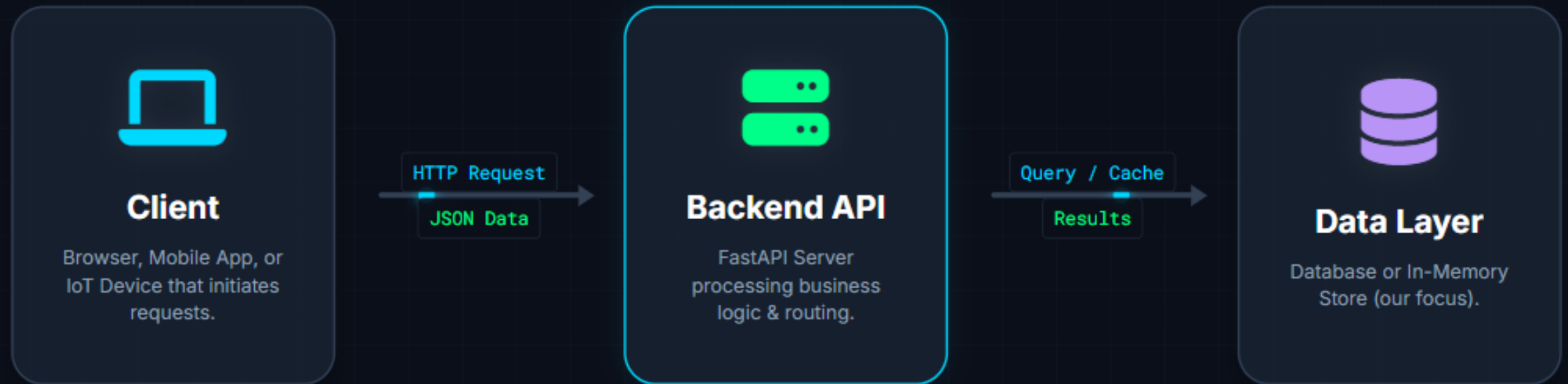
Server-side logic that powers applications. It's invisible to the user but handles all the heavy lifting.



Data & Logic Central

Manages data storage, complex calculations, business rules, and integrations with third-party services.

Client-Server-Database Architecture



What is an API?



The Contract

Application Programming Interface: A formal agreement between client and server defining how they communicate and exchange data.



The Components

Defined by specific **Endpoints** (URLs), **Methods** (actions like GET/POST), **Inputs** (parameters), and **Outputs** (responses).



Decoupling Benefits

Separates frontend from backend. You can rewrite the backend logic completely without breaking the client app if the API remains the same.



HTTP + JSON

Modern Web APIs predominantly use the HTTP protocol for transport and JSON (JavaScript Object Notation) as the universal language.

API Styles & When to Use Them



REST

RESOURCE

Standard architecture using HTTP verbs (GET, POST) to manipulate resources.

USE WHEN: Public APIs, Simple Web Services, Caching

💡 **Note:** We will focus on **REST** with **FastAPI** as it's the foundation of modern web development.



GraphQL

QUERY

Client-driven query language. Fetches exactly what is needed in a single request.

USE WHEN: Complex Frontends, Reducing Over-fetching



Webhooks / Sockets

EVENT

Real-time event notification (push) or bi-directional streaming channels.

USE WHEN: Live Chat, Notifications, Real-time Updates

HTTP Methods (CRUD)



GET

POST

PUT

DEL

GET

Read Resource

Retrieves data from the server. Should not modify state. Safe to call multiple times without side effects.

POST

Create Resource

Submits data to create a new resource. Calling it multiple times creates multiple resources.

PUT

Replace Resource

Updates an entire resource. If it doesn't exist, it can create it. Sending the same data twice has no additional effect.

DEL

Delete Resource

Removes a resource. Calling it multiple times should result in the same state (resource gone), though status codes may vary (200 vs 404).

Idempotent

HTTP Status Codes

RANGE	CATEGORY	MEANING	COMMON EXAMPLES
200	Success	The action was successfully received, understood, and accepted.	200 OK
4xx	Client Error	The request contains bad syntax or cannot be fulfilled. It's the client's fault.	400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found
5xx	Server Error	The server failed to fulfill an apparently valid request. It's the backend's fault.	500 Internal Error 502 Bad Gateway 503 Unavailable

Request & Response Anatomy

CLIENT REQUEST

```
curl -X POST 'http://api.local/items?limit=10' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "FastAPI Book",
    "price": 19.99
  }'
```



SERVER RESPONSE

```
{
  "id": 1,
  "name": "FastAPI Book",
  "price": 19.99,
  "tags": []
}
```



The HTTP Request

METHOD (VERB)

POST

ENDPOINT (URL)

/items

HEADERS

Content-Type

QUERY PARAMS

?limit=10

The client initiates action. The **Body** contains the data (payload) to be processed, usually in JSON format.



The HTTP Response

STATUS CODE

201 Created

MEDIA TYPE

application/json

The server replies with a status code indicating success or failure, and returns the created resource or requested data.

REST API Best Practices



Resource Naming

Use plural nouns for resources, not verbs.
Represent hierarchy with path segments.

`GET /users/123/orders`



Proper Status Codes

Don't always return 200. Use standard codes to communicate outcome semantics.

`201`

`400`

`404`



Validation & Types

Validate all inputs strictly. Fail fast with clear error messages (422 Unprocessable).

`Pydantic Models`



Pagination & Filtering

Never return full datasets. Protect the database with default limits and offsets.

`?limit=10&offset=0`



API Versioning

Future-proof your API. Changes shouldn't break existing clients.

`/api/v1/items`



Performance	A+
Security	A
Reliability	A+
Docs	100%



Backend Architecture Choices



Monolith

Unified Codebase

- ✓ Single repository containing all logic (Auth, Products, Orders).
- ✓ Simple to develop, test, and deploy initially.
- ✓ Direct in-memory function calls (very fast).

Best For: Small Teams, Early Stage

Simple Ops

Shared Memory



Microservices

Distributed Systems

- ✓ Split into independent services communicating via Network.
- ✓ Scale specific parts (e.g., scale "Search" but not "Auth").
- ✓ High operational complexity (Service Discovery, Tracing).

Best For: Large Orgs, High Traffic, distinct domains

Scalable

Complex Ops

Internal Layering

Structure within a service



API Layer

Routes & Schemas



Service Layer

Business Logic



Data Layer

Storage / DB

Why FastAPI?



High Performance



On par with NodeJS and Go. Powered by **Starlette** (web) and **Pydantic** (data validation).

Modern & Async-First



Native support for `async/await`. Handles high concurrency easily while remaining compatible with standard libraries.

Automatic Documentation



Generates interactive API docs (Swagger UI & ReDoc) automatically from your code standards.

Install & Project Setup

TERMINAL

```
# 2. Install FastAPI & Uvicorn  
$ pip install "fastapi[standard]"
```

RUN SERVER

```
# Run development server with hot reload  
$ uvicorn app.main:app --reload  
INFO: Uvicorn running on http://127.0.0.1:8000
```



Environment & Dependencies

Always work inside a **virtual environment** to isolate dependencies. We install `fastapi` for the framework and `uvicorn` as the ASGI server.



Python 3.8+ Required



[standard] includes `uvicorn`



Project Structure

```
my-project/  
├── app/  
    └── main.py
```

The `--reload` flag enables hot reloading, so the server restarts automatically whenever you save changes to `main.py`.

Path Parameters

MAIN.PY

```
from fastapi import FastAPI, HTTPException
app = FastAPI()

@app.get("/users/{user_id}")
def read_user(user_id: int):
    if user_id <= 0:
        raise HTTPException(
            status_code=400,
            detail="Invalid ID"
        )
    return {"user_id": user_id}
```

SAMPLE USAGE

```
# Valid Request (200 OK)
GET /users/42 → {"user_id": 42}

# Validation Error (422 Unprocessable)
GET /users/abc → {"detail": "value is not a valid integer"}
```

Declaration & Typing

SYNTAX `/users/{user_id}`

TYPE HINT `user_id: int`

FastAPI uses standard Python type hints to declare path parameters. The value in the curly braces {} matches the function argument name.

Smart Features



Data Conversion

String "42" becomes Integer 42



Auto Validation

Rejects non-integer inputs automatically

Request Body with Pydantic

```
SCHEMAS.PY

from pydantic import BaseModel

class Item(BaseModel):
    id: int
    name: str
    price: float
    tags: list[str] = []
```



```
MAIN.PY

@app.post("/items")
def create_item(item: Item):
    # JSON body is parsed & validated
    return item
```



Why use Pydantic?



Automatic Validation

FastAPI automatically validates incoming JSON against your model. Incorrect types trigger clear 422 errors.



Data Conversion

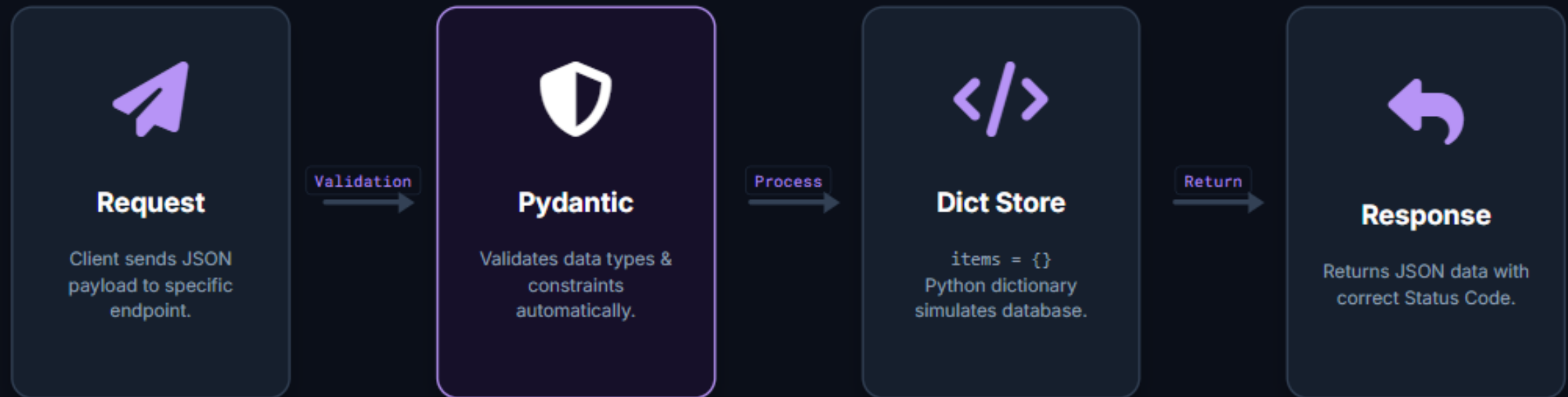
Input data is converted to Python types (e.g., JSON string "10.5" becomes Python float 10.5).



JSON Schema Support

Models are used to generate OpenAPI (Swagger) schemas automatically, making API docs interactive.

CRUD (In-Memory, No DB/ORM)



📑 Endpoints

Implement standard CRUD operations: Create (POST), Read (GET), Update (PUT/PATCH), Delete (DELETE).

🗄️ Data Store

Using a simple global variable `items: Dict[int, Item]` to store state in memory during runtime.

⚠️ Validation

Pydantic models ensure incoming data matches the schema before it ever reaches our logic function.

📋 Status Codes

Explicitly set codes like 201 Created or 204 No Content to adhere to REST standards.

Create & Read Endpoints

```
1  from typing import Dict
2  from fastapi import HTTPException
3
4  # Using a dictionary as our "database"
5  items: Dict[int, Item] = {}
6
7  @app.post("/items", status_code=201)
8  def create_item(item: Item):
9      if item.id in items:
10         raise HTTPException(status_code=400, detail="ID already exists")
11         items[item.id] = item
12         return item
13
14  @app.get("/items/{item_id}", response_model=Item)
15  def get_item(item_id: int):
16      item = items.get(item_id)
17      if not item:
18         raise HTTPException(status_code=404, detail="Item not found")
```

 In-Memory Store

 Tip: **201 Created** is the standard status for successful creation.

Response Models & Status Codes

```
APP/MAIN.PY

from pydantic import BaseModel
from fastapi import FastAPI

# Define output schema (hides internal fields like 'password')
class ItemOut(BaseModel):
    id: int
    name: str
    price_with_tax: float

@app.post(
    "/items",
    response_model=ItemOut
)
def create_item(item: Item):
    # Logic to save item...
    return item
```

Controlling Responses

✂ response_model

Automatically filters data. Use this to hide sensitive fields (e.g., passwords) or format output (e.g., converting ORM objects to JSON).

✓ status_code

Sets the default HTTP status for success. Common choices: 201 Created, 204 No Content.

📖 responses

Adds documentation for additional status codes (like 400 or 404 errors) to the auto-generated Swagger UI.

💡 **Pro Tip:** Even if your function returns a database object with extra fields, `response_model` ensures the client only sees what you explicitly defined in the schema.

Auto Documentation

```
from fastapi import FastAPI

# Define API Metadata
app = FastAPI(
    title="Section 2 API",
    version="1.0.0",
    description="Backend for Class",
    docs_url="/docs"
)

@app.get("/items")
def read_items():
    return [{"name": "Item 1"}]
```

💡 Type hints in your code (Pydantic models) automatically generate the schemas seen in these docs.



Swagger UI

URL <http://localhost:8000/docs>

Interactive documentation that lets you call your API directly from the browser. Great for testing and debugging endpoints instantly.

Backend Meets AI: Why Integrate?



Enhanced Features

Power next-gen UX with conversational interfaces, personalized recommendations, and smart content summarization.



Automated Workflows

Streamline operations via intelligent document extraction, sentiment classification, and RAG pipelines.



Exposing Capabilities

Wrap complex ML models or LLMs behind clean, secure REST APIs for frontend or mobile consumption.



Key Considerations

Balance trade-offs in latency, inference costs, data privacy, and the need for new observability metrics.

AI Integration Patterns



External API

e.g., OpenAI, Anthropic, Cohere

👍 ADVANTAGES

- ✓ **State-of-the-Art:** Immediate access to the most powerful models (GPT-4, Claude).
- ✓ **No Infrastructure:** Zero operational overhead for hosting heavy models.

⚠️ CHALLENGES

- ✗ **Latency & Cost:** Network hops add delay; per-token pricing scales with usage.
- ✗ **Data Governance:** Sending sensitive data to third-party providers.



Local Serving

e.g., Llama 3, Mistral, BERT

👍 ADVANTAGES

- ✓ **Control & Privacy:** Data never leaves your infrastructure; full fine-tuning control.
- ✓ **Offline & Predictable:** Works without internet; fixed infrastructure costs.

⚠️ CHALLENGES

- ✗ **Infrastructure Load:** Requires GPU resources and careful scaling management.
- ✗ **Lifecycle Ops:** Complex MLOps for deployment, versioning, and monitoring.

Streaming AI Responses (SSE)

```
from fastapi.responses import StreamingResponse
import time

def generate_tokens(prompt: str):
    # Simulate LLM streaming logic
    for token in ["Hello", " ", "from", " ", "AI!"]:
        time.sleep(0.1)
        yield f"data: {token}\n\n"

@app.get("/ai/stream")
def ai_stream(prompt: str):
    return StreamingResponse(
        generate_tokens(prompt),
        media_type="text/event-stream"
    )
```

⚡ Why Stream?

Large Language Models (LLMs) take time to generate full responses. Streaming sends tokens as they are generated, drastically improving **User Experience (UX)** by reducing perceived latency.

🔄 Server-Sent Events (SSE)



MIME TYPE
text/event-stream



MECHANISM
Python Generators (yield)

Standard HTTP for unidirectional real-time data. The client keeps the connection open while the server pushes updates.

Serve a Local ML Model

```
import joblib
from pydantic import BaseModel

# 1. Load artifacts at startup (Global Scope)
model = joblib.load("model.pkl")
vectorizer = joblib.load("vec.pkl")

# 2. Define Input Schema
class TextIn(BaseModel):
    text: str

# 3. Predict Endpoint
@app.post("/ml/predict")
def predict(inp: TextIn):
    # Preprocessing
    X = vectorizer.transform([inp.text])

    # Inference
    y = model.predict(X)[0]

    return {"label": int(y)}
```

APP/ML.PY



Implementation Strategy



Load on Startup: Load heavy models like .pkl files or pytorch weights once when the app starts, not inside the request loop.



Global Scope: Keep model variables global so they persist in memory across multiple requests.

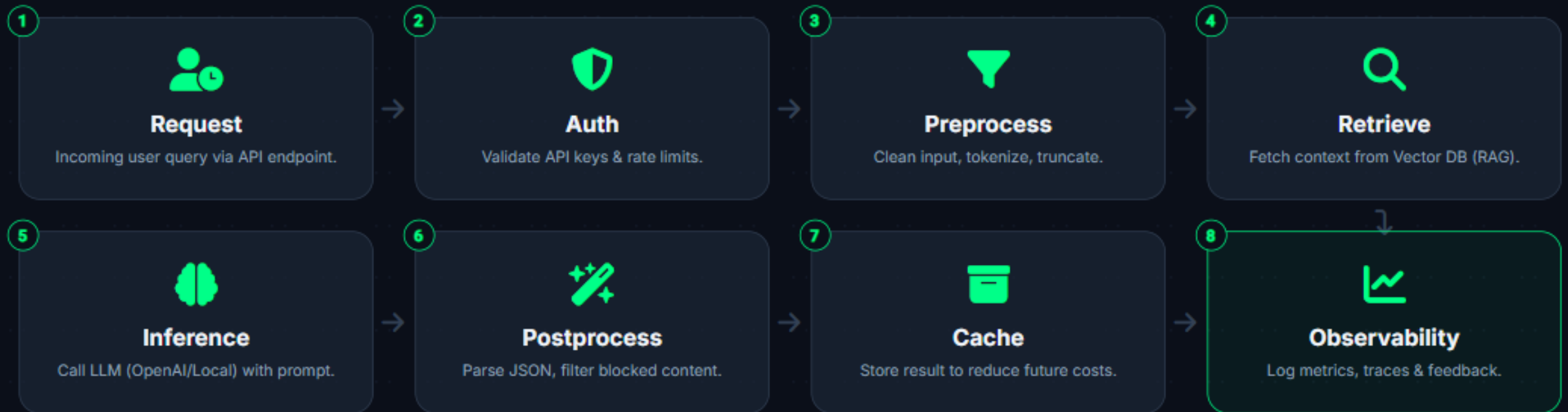


Strict Validation: Use Pydantic models (TextIn) to ensure the input data matches what the model expects before inference.



Thin Endpoints: Keep the endpoint logic minimal—focus on preprocessing input, running inference, and formatting output.

AI Pipeline Architecture



⌚ Reliability

Implement strict timeouts, retries with backoff, and circuit breakers to handle external LLM latency/failures.

🔒 Safety & Privacy

Sanitize logs to remove PII (Personally Identifiable Information) before storing in observability tools.

📊 Metrics

Track token usage (cost), end-to-end latency, and response quality for optimization.

Summary & Next Steps



Mission Accomplished

- **Backend Fundamentals**
APIs, HTTP Methods, REST Architecture
- **FastAPI Construction**
Path/Query Params, Pydantic, CRUD, Docs
- **AI Integration**
External LLMs, Local Models, Streaming



Future Missions

- **Persistence Layer**
Connect real Databases (PostgreSQL) & ORMs
- **Security & Auth**
Implement OAuth2, JWT Tokens, and Permissions
- **Production Readiness**
Testing (Pytest), Docker, CI/CD, RAG Apps